[23] If implemented correctly: in particular, you need to be careful how you break ties when two entries have the same count – assigning higher value to newer entries is usually the best choice.
[24] One could say a FIFO+ queue, since elements can also be moved to the head of the queue at any time.
[25] Depending on the relative frequency of read/write, you might want to fine tune one operation or the other.
[26] This kind of cache is particularly useful in *eventually consistent* systems. Eventual consistency relaxes the consistency constraint, allowing for instance data in the cache to be slightly off-sync with respect to the most recent version of the same entry stored in a database. This can be useful if we are fine with some info being slightly off-sync: for instance, for a shopping cart, we probably can handle the availability of an entry to be off-sync for a 100ms or even a second.
[27] Or, in other words, they can be safely run in a multi-threading environment, without leading to race conditions.
[28] Java standard library doesn't provide an implementation of `PriorityQueue`, so you won't find this class natively in your JRE. We provide a version of a synchronized heap in our repo.
[29] A situation where all threads are blocked waiting for some shared resources, but none of them can acquire it: this causes the application to become unresponsive, stuck and ultimately crash.
[30] A superb start is C++ Concurrency in Action, if you are familiar with C++, or would like to learn more about it. Concurrency in .NET, is your choice of election if functional programming is your way.
[31] If thread A locks on the hash table and thread B acquires the write lock on the whole cache, and then both need the other lock to continue, we get to the equivalent of a Mexican standoff for threads. And never ends well in movies.
[32] You can still have race conditions, when 2 threads try to update the shared cache at the same time, but this case, can be solved with *optimistic locks*, basically (in an extreme simplification) versioning your resource. For further insight about optimistic locks, you might take a look at Compare-and-swap (*CAS*) algorithm.
Using immutability + optimistic locks you get a sensible performance boost, in most common contexts/languages.
[33] Object-Oriented, languages like Java were classes and objects are language basic building blocks.
[34] Consistent hashing is a special kind of hashing that guarantees that, when one of the nodes is removed, only the keys hosted on that node needs to be remapped. See figure 7.14 to get an idea of how it works. More in general, if applied to a hash table, consistent hashing guarantees that when a table with n slots is resized, only one n-th of the keys stored needs to be remapped. This is particularly important for distributed caches, where billion of elements might need to be remapped otherwise, possibly causing the whole cache to crash.